Where do we use YAML files in devops?

Kubernetes objects is written in yaml file format.

ansible playbooks is written in yaml file format.

docker compose is written in yaml file format.

Gitlab cicd pipeline file will be written in yaml file format.

NOTE:

Before starting with k8s, we have to **assume** that the following have been setup already,

- Code for Application has been written, Dockerfile was also written & everything (artifacts, libraries, dependencies) needs to run application has been packed into Docker Images
- That Docker Images are available on a Docker repository like Docker hub, so Kubernetes can pull it down.

Ultimate Aim?

Final goal is to deploy an application as containers, but on a multiple servers that are configured as worker nodes in a cluster.

Kubernetes does not deploy containers directly on the worker nodes.

(i.e. Kubernetes ***cannot create container directly***, so it will create an object called as pod) pod in-turn can create container. (containers will be running inside the pod)

IMPORTANT

Workflow of pod creation:

In order to deploy our containerized application in k8s

we need to create a pod ==>container==>application

To create a pod, we need to write manifest (yaml) file

Once we have manifest file ready, we request to kube-api server to create pod

Step1: Kube-api server checks the manifest file & stores pod (container) related information in etcd

also forwards pod creation request to kube-scheduler

Step2: Kube-Scheduler will selects the best worker node & asks kubelet (agent) in selected worker node to create pod.

Step3: Kubelet will communicate with container run time (docker) pulls the image & creates container inside the pod.

Step4: once pod gets created, controller-manager will periodically check if the pod is running or not, in case if the pod gets crashed / exited

Controller creates a new Pod automatically in order to match the desired state.

What are objects in Kubernetes?

Objects help in achieving all features of containers orchestration

eg: pod, replicaset, deployment, secrets, services etc

Objects can be created using YAML files

Kubernetes uses various types of objects.

1. Pod:

Pod is the basic object in Kubernetes.

A ***POD*** is a single instance of an application.

Inside the pod, we will have the containers running.

Note: *kubectl* commands will work on the pod! pod *passes* on those instructions to the container.

2. Service Object:

Service object is used for port mapping and network load balancing.

3. NameSpace:

This is used for creating logical partitions inside the k8s cluster.

4. Secrets:

This is used for storing & passing encrypted data to the pods.

5. ReplicaSet / Replication Controller:

ReplicaSets are used for managing multiple replicas of a pod to perform activities like load balancing and autoscaling.

6. Deployment:

Deployment is used for performing all activities that a ReplicaSet can do. It can also handle rolling updates.

Ways to Create Kubernetes (K8s) Objects?

1. **Declarative way**
   - ❖ Write manifest files (YAML files) & apply these manifest files.
   - ❖ YAML files used in k8s are also called as definition files (or) manifest files.
   - ❖ YAML file-based creation of k8s objects are preferred as we can keep files used for objects creation.

2. **Imperative way (direct command)**
   - ❖ Directly creates objects from the command line.
   - ❖ creating k8s objects from cli are not best practice
   - ❖ this can be preferred only if we want to create a k8s object quickly.

What is Pod?

A ***POD*** is the basic ***object*** that users can create in Kubernetes.

Inside the pod, we will have the containers running.

How to create Pod **Imperative way** (Directly from command)?

syntax: `kubectl run <pod_name> --image <image_name>`

To create container in docker:

```
$ docker run -- name <containerName> <imagename>
```

To list all pods:

```
$ kubectl get pods
```

To know on which worker node, pod is running

```
$ kubectl get pods -o wide
```

To list all nodes in a cluster:

```
$ kubectl get nodes
```

To delete any pod:

```
$ kubectl delete pod <pod-name>
```

How to write manifest files files in k8s ?

Kubernetes uses YAML files as input for the Creation of objects such as *PODs, Replicasets, Deployments, Services* etc

All manifest files (yaml files) in k8s will contain **4 top level elements**,

All of these acts as siblings (like children of the same parents)

All 4 below elements are REQUIRED fields, so all 4 must be available in any k8s manifest file

Pod-manifest.yaml

```
apiVersion:
kind:
metadata:


spec:
```

1. **apiVersion:**
2. **kind:**
3. **metadata:**
4. **spec:**

1. apiVersion:

Depending on type of Kubernetes object we want to create, there is corresponding code library we want to use.

Kind	apiVersion
Pod	v1
Replication Controller	v1
Service	v1
Namespaces	v1
Secrets	v1
ReplicaSets	apps/v1
Deployment	apps/v1

2. kind:

Kind Refers to Kubernetes object which we are trying to create.

Ex:

- for Pod --> `kind: Pod`
- for Replicaset object --> `kind: Replicaset`
- for service object --> `kind: Service`

object name in kind, will start with upper case letter, in -> kind: Pod (P is uppercase)

3. Metadata:

The metadata is like Additional information about the Kubernetes object getting created

Eg: object name, labels etc.

Note: Labels are user defined as key-value pairs, labels helps in organizing k8s objects

4. spec (specifications):

Contains container related information like image name, environment variables, port mapping etc.

spec block

```
spec:
  containers:
  - name: <name_of_container>
    image: <name_of_image>
```

To create a pod with a single container.

Example 1: Write a manifest file to deploy tomee docker image

pod-name: my-pod-1

container name as c1

also include labels as author: bharath

```
---
apiVersion: v1
kind: Pod
metadata:
  name: my-pod-11
  labels:
    author: bharath
spec:
  containers:
  - name: c1
    image: tomee
 ...
```

kubectl apply -f <manifest-file>.yaml

Check pods created using **$ kubectl get pods**

Example 2: Write a manifest file to deploy nginx docker image

pod-name as ***my-pod-2***

container name as ***my-container-1***

also include labels as ***author*** as ***bharath*** & ***place*** as ***Bangalore***

```
---
apiVersion: v1
kind: Pod
metadata:
  name: my-pod-2
  labels:
    author: bharath
    location: bangalore
spec:
  containers:
    - name: my-container-1
      image: nginx
...
```

kubectl apply -f <manifest-file>.yaml

Check pods created using **$ kubectl get pods**

To create a multi container pod with 2 containers.

Example 3: Write a manifest file to deploy nginx & redis docker images

pod-name as ***my-multicontainer-pod***

first container name as ***my-nginx-container*** & image ***nginx***

second container name as ***my-redis-container*** & image ***redis***

also include labels as ***author*** as ***bharath*** & ***place*** as ***Bangalore***

```yaml
---
apiVersion: v1
kind: Pod
metadata:
  name: my-multicontainer-pod
  labels:
    author: bharath
    location: bangalore
spec:
  containers:
    - image: nginx
      name: my-nginx-container
    - image: redis
      name: my-redis-container

...
```

kubectl apply -f <manifest-file>.yaml

Check pods created using **$ kubectl get pods**

Check pod with 2 containers created

```
root@k8s-master:~# kubectl get pods
NAME                   READY   STATUS    RESTARTS   AGE
my-multicontainer-pod   2/2     Running   0          6s
```